

Deploying Windows 8: Unattended Installation Using Windows Deployment Services (WDS) – Part I

10–13 minutes

See also: [Deploying Windows 8: Unattended Installation Using Windows Deployment Services \(WDS\) – Part II](#)

For a while now I've been preparing articles in this blog regarding deploying operating systems, and especially unattended deployments. I've started with Windows Vista deployments and the initial version of Windows Deployment Services ([Post I](#), [Post II](#) and [Post III](#)); the moving forward with Windows 7 using also WDS ([Post I](#) and [Post II](#)), and also MDT 2010 adding Microsoft Office 2010 to the mix ([Post I](#), [Post II](#) and [Post III](#)).

And of course, with the release of Windows 8 I had the necessity to prepare a new set, even though this new OS is just at "Consumer Preview" beta.

This set of posts is intended to provide **step-by-step procedures to accomplish a full automated deployment of Windows 8 using WDS**.

Overview of the Process

The main goal of this process we are going to review in detail is to provide an **unattended deployment of Windows 8 using Windows Deployment Services (WDS)**.

In order to provide a complete guidance, we are going to **capture an installed Windows 8 image, add it in WDS and then use this customized image to deploy it using unattended files**.

In this set of posts we are going to examine:

1. **Reviewing requirements.**
2. **Installing and configuring WDS Role.**
3. **Adding Boot and clean Windows 8 images to WDS.**
4. **Create a capture boot image in WDS.**
5. **Capture the Windows 8 reference machine.**
6. **Prepare Windows System Image Manager (WISM).**
7. **Manage first unattended file: WDSClientUnattend.xml**
8. **Manage second unattended file: AutoAttend.xml**
9. **Deploy Windows 8 using Unattended Files.**

1. Reviewing Requirements

For the scenario we are going to review, here are the pre requisites we must fulfill in order to complete the unattended deployment of Windows 8:

- **Active Directory and DNS server in place.** The computer running WDS must be a member of an Active Directory.
- An active **DHCP** server on the network
- An **NTFS partition** on the server with the WDS role to store your OS images.
- **Windows 8 media.** To download it, access this link “[Windows 8 Consumer Preview](#)”.
- **Windows Automated Installation Kit (WAIK).** This is an optional component that we can use to create unattended files.

The WAIK version we are going to use is the version released for Windows 7, which is available in the following link: “[The Windows® Automated Installation Kit \(AIK\) for Windows® 7](#)”

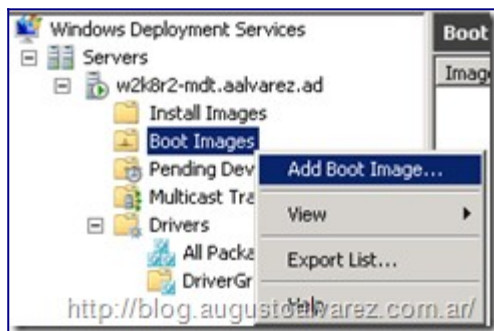
2. Installing and Configuring WDS Role

The steps necessary for this are really simple and covered in this previous post of mine: “[Deploying Windows 7 Using Windows Deployment Services \(WDS\): Step-by-Step – Part I](#)”.

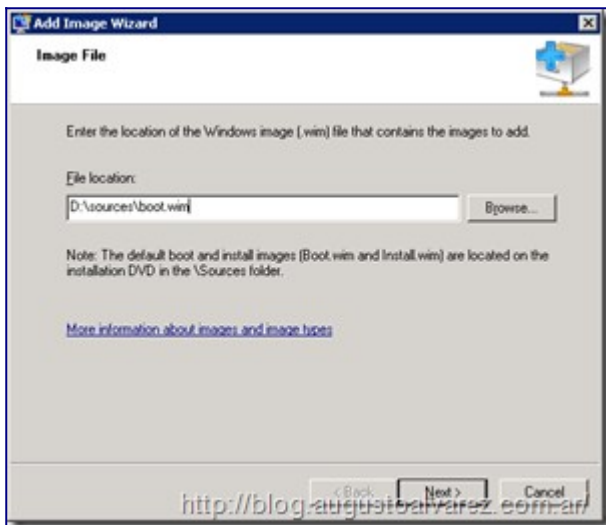
3. Adding Boot and Clean Windows 8 Images to WDS

These steps are also simple if you ever manage WDS. Let’s review them:

3.1. Access the WDS Console and right click in “Boot Images” and select “Add boot image”.

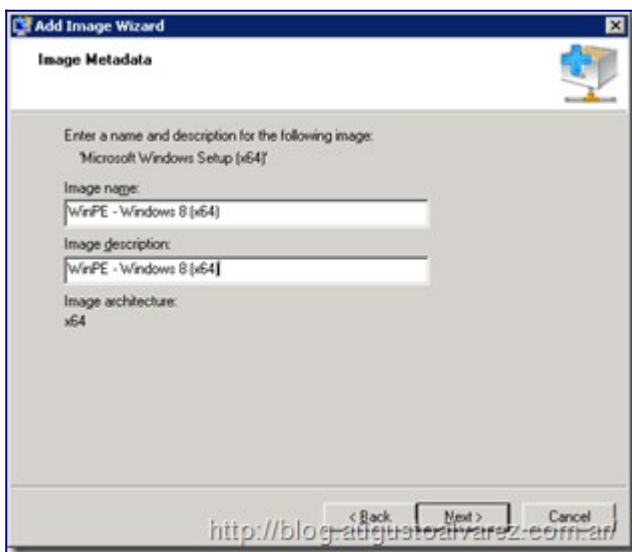


3.2. Select the Windows 8 media and locate “%drive%\sources\boot.wim”.



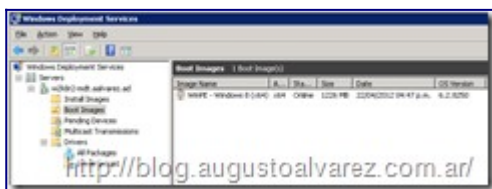
The “boot.wim” file is the Windows Preinstallation Environment (Windows PE), which is a minimalistic OS in charge of preparing the environment prior the installation. For more information about Windows PE, check the following link: [“What is Windows PE?”](http://blog.augustoalvarez.com.ar/2012/07/what-is-windows-pe/)

3.3. Select the name we are going to use to identify the image, in my case “WinPE – Windows 8 (x64)”.

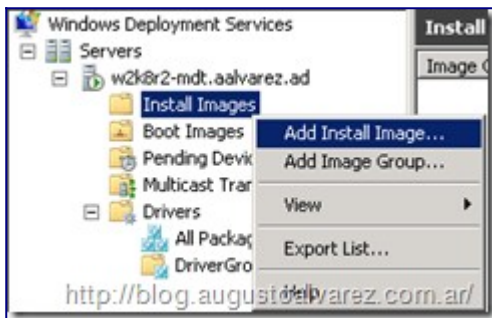


3.4. Click “Next” in the following screen and wait until the process completes.

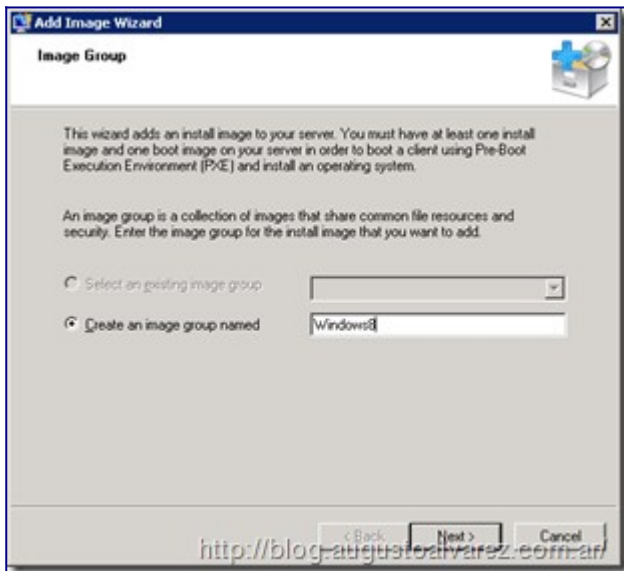
3.5. Once it’s completed, in the WDS console the new image will appear.



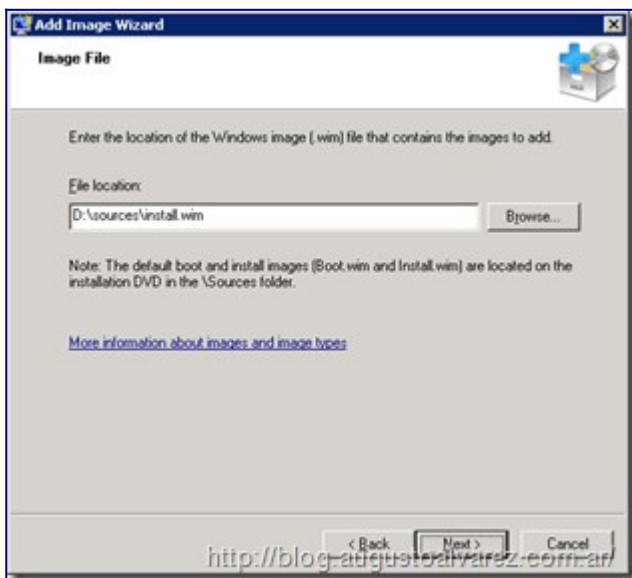
3.6. Now, for adding a clean Windows 8 install image, in the WDS console right-click “Install Images” and select “Add Install Image”.



3.7. Create a new image group; in my case I'm using "Windows8".



3.8. Select the installation file available in Windows 8 media in the following path: "%drive%\sources\install.wim".

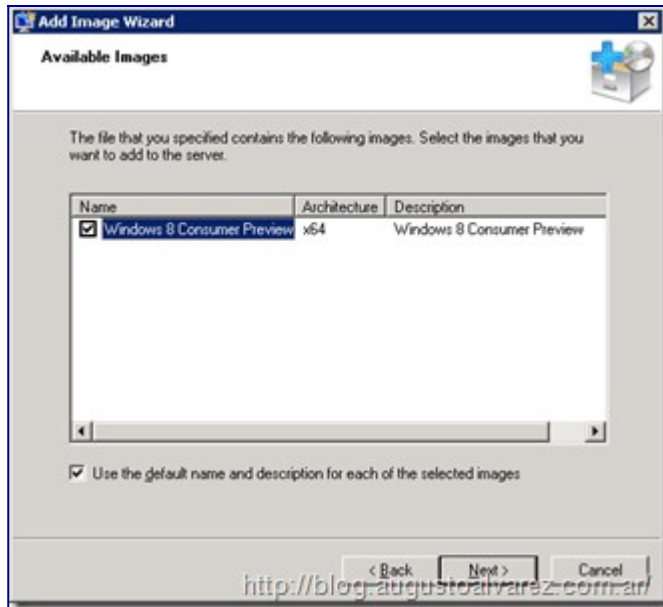


The "install.wim" file is where all the Windows operating system files are stored and compressed for optimization. In this new version, the Windows 8 files we have available in the media correspond to the specific version of Windows we've downloaded. In the

previous versions (Windows Vista and Windows 7) the size of “install.wim” was a little bit bigger since it contained files from several OS versions.

Also, in Windows 8, the compression algorithm for this file has change in order to optimize the download process for installation.

3.9. Click “Next” in the image selection window.



3.10. Also click “Next” in the following step to confirm the new image to be added.

3.11. Once we click “Finish”, we will see the new image added to Windows Deployment Services.

An important part of the OS life cycle in our environment is to distinguish the base images we are using. This base image does not only refer to the OS, it also includes the applications and basic configurations we add to the operating system.

Even though we don’t need to capture a reference Windows 8 image to accomplish an unattended deployment, we are going to take this scenario to understand a little bit more about the possibilities in WDS.

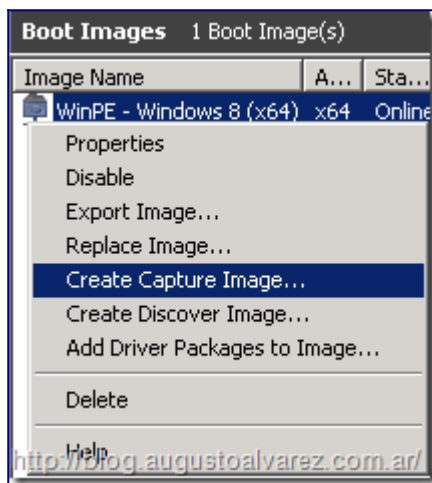
4. Create a Capture Boot Image in WDS

In order to be able to capture a Windows 8 image using WDS, we must convert a boot image into a “Capture Boot Image”.

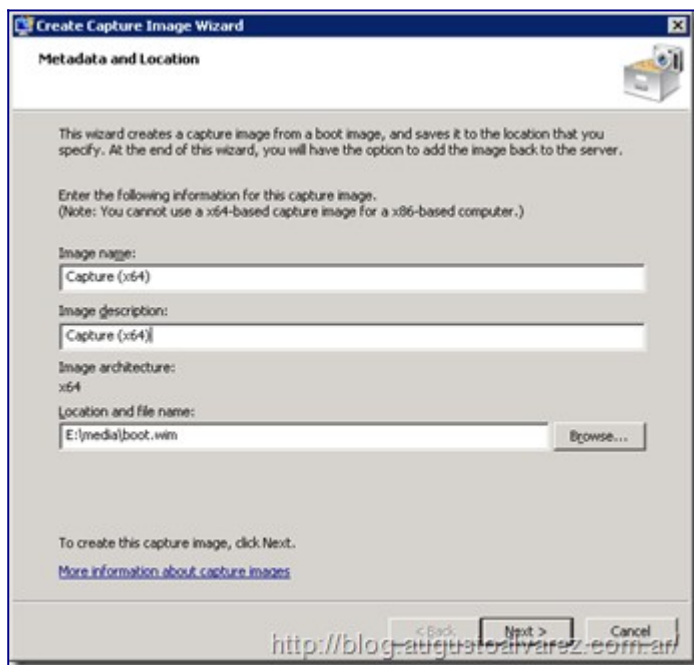
This type of image is basically a Windows PE modified to be in charge of capturing a prepared Windows 8 image, instead of providing the environment for an installation. The steps are really simple:

4.1. Copy the “boot.wim” file located in Windows 8 media to a local folder in WDS. Alternatively we can “Export” the boot image we’ve added in WDS.

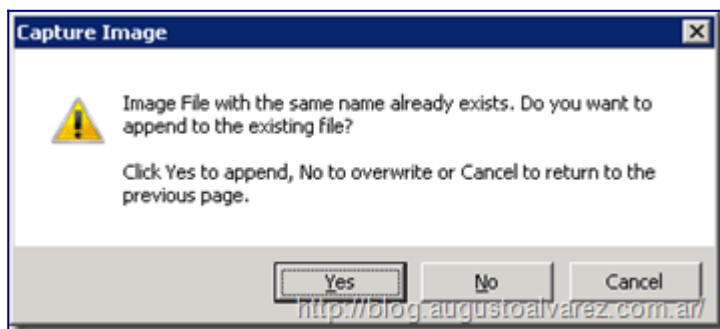
4.2. Right-click in the existing boot image in WDS and select “Create Capture Image”.



4.3. Select the image name, description, and the location of the “boot.wim” file copied in the first step.



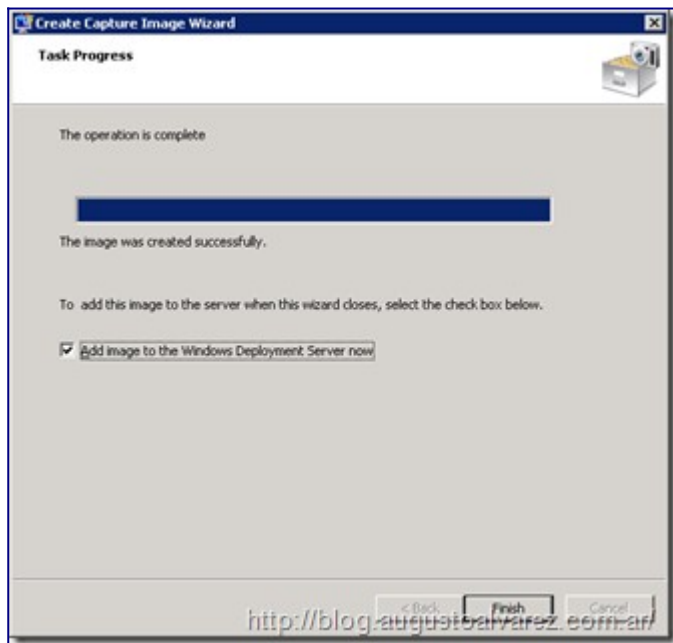
4.4. We’ll receive a warning regarding the image we are about to modify: “Image File with the same name already exists. Do you want to append to the existing file?”



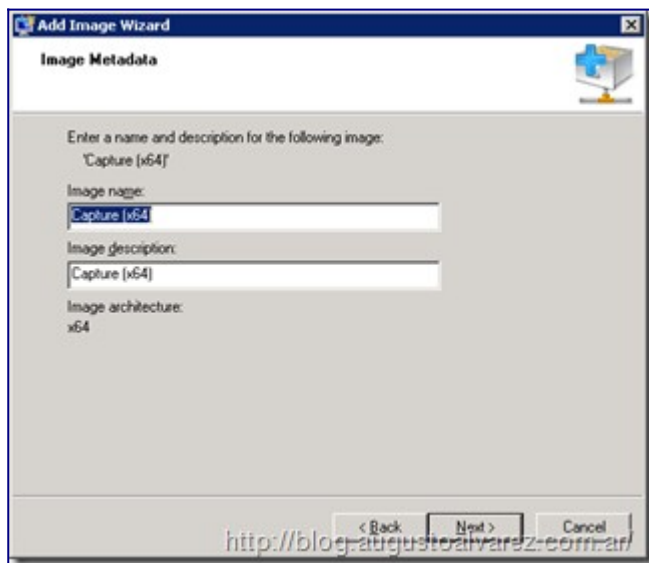
Clicking “Yes” will keep the original WinPE image and add the capture WinPE; if we click “No”, the entire image will be overwritten with the new capture WinPE.

Take note that we are not going to replace the boot image we've added earlier in WDS; in these steps we are going to generate a new one.

4.5. Complete the steps and select “Add image to the Windows Deployment Server now”.



4.6. Confirm the location of the new boot.wim file.



4.7. Insert the boot image name, in my case “Capture (x64)”.

4.8. Complete the wizard and the new boot image should be added to WDS.

5. Capture the Windows 8 Reference Machine

The capturing process of a Windows image is performed by using a tool provided by Microsoft “**sysprep**”. Sysprep is the component Microsoft provides in order to IT teams be able to scale OS

implementations.

With sysprep we are able to “**generalize**” an OS by removing some particular characteristics in Windows and transforming this new “generalized” image in a prepared OS ready for a massive deployment. If the image is not generalized we will not be able to capture it using the image we’ve selected earlier.

By applying sysprep we’ll be able to **remove** some specific data from the OS installation, for example:

- **Computer name**
- **Owner and Company name**
- **SID**
- **Domain or workgroup membership**
- **TCP/IP Settings**
- **Regional and keyboard settings**
- **Specific hardware drivers.** This refers to specific computer hardware, like video or audio drivers. But if you only applied drivers used on the Windows installation, the same will apply for the deployment, but any other external driver installed will be unavailable.
- **Any saved network connections** (wireless networks saved)
- **OS product key.** This is an important note, since no matter if your product has been activated; the key is reset after this process.

But of course, there are other several components that will be **kept** to maintain this as a base image. For example:

- **Programs and features installed.**
- **Local Users and Groups created.**
- **Product Keys used for programs installed.** Meaning if you have Microsoft Office installed, the key used will remain as the same on the deployments.
- **Windows updates installed**
- **User Profiles:** Since all the profiles configuration are basically data stored on the Users folders, all that information will be uploaded within the image.
- **Printers installed.**

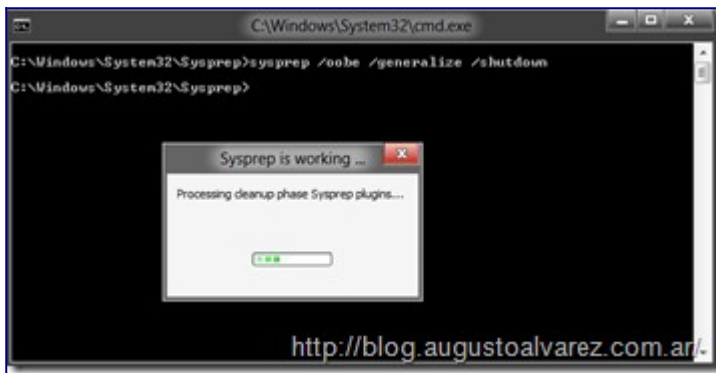
For more information about sysprep, take a look to the following link: “[What Is Sysprep?](#)”

The process of preparing the image requires only running a specific command line; let’s take a look to the steps:

5.1. Access the reference Windows 8 image.

5.2. In a command prompt, locate the following folder: “*C:\Windows\system32\sysprep*”.

5.3. Run “**sysprep /oobe /generalize /shutdown**” (alternatively the */reboot* parameter can be used). This command line will generalize the OS removing the specific components mentioned above and shutdown after the process is completed.

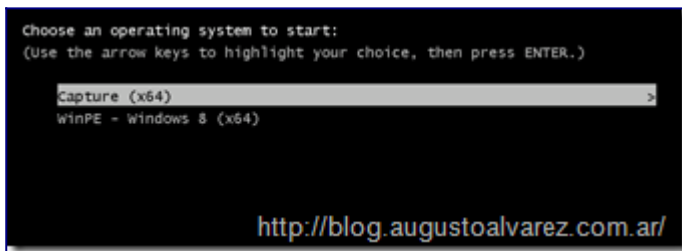


We must remember that, since this OS is going to be generalized, the next the normal boot process occurs, it will appear as Windows was just installed. And, of course, prior to that we must capture this image and store it as our base Windows 8 image in WDS.

For more information about this generalize process, take a look to the following link:
[“Prepare to Capture an Image for Deployment \(Generalize\)”](http://blog.augustoalvarez.com.ar/prepare-to-capture-an-image-for-deployment-generalize)

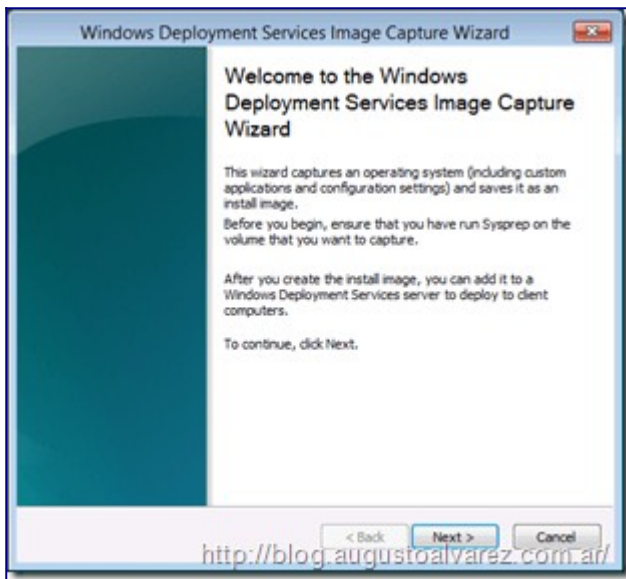
5.4. Configure the reference machine to boot from network. Usually by using F12 in the start of the boot process we will be able to select the network card as the preferred boot option.

5.5. Boot from network in the reference machine. Since we have two boot images available in WDS, we should see something like this.

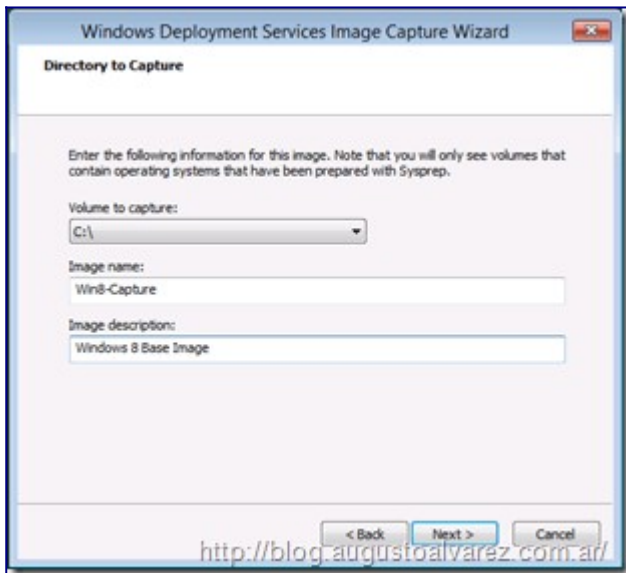


Select the capture image.

5.6. Once the capture image is loaded, a new wizard will appear.

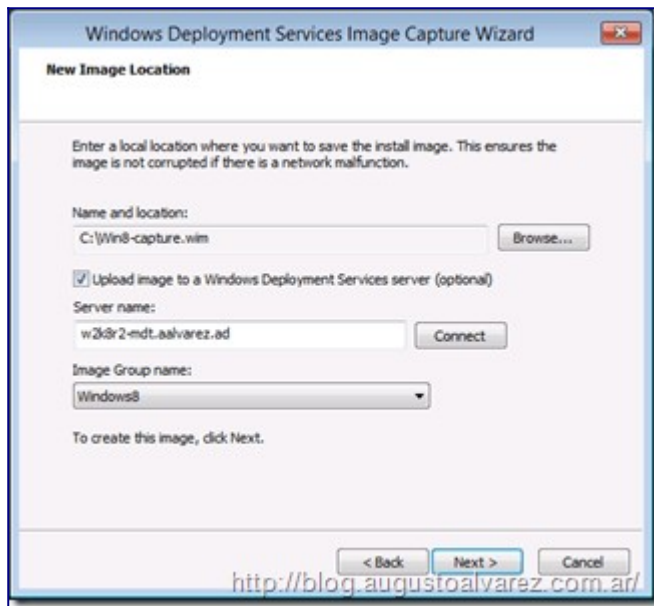


5.7. Select the volume to capture “C:\” and select an image name and description.



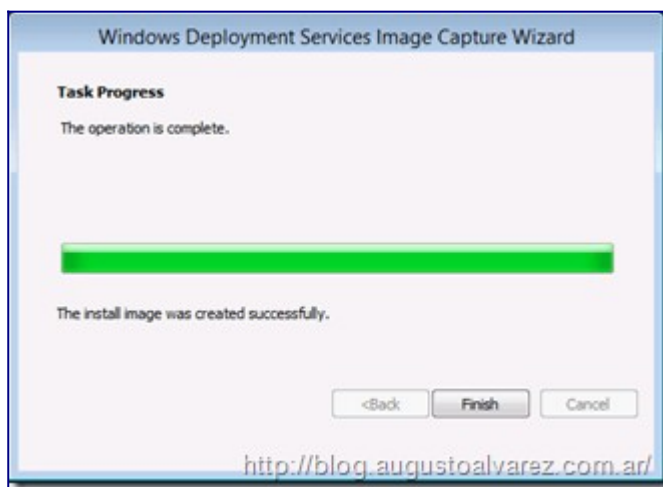
If no drive is available to select, this means that the image was not generalized properly. **The only volumes that can be captured are those generalized with sysprep.**

5.8. Select a local path for the image location and optionally we can upload the image we are going to create to WDS; for that we need to use the WDS Server FQDN (we will be prompt for credentials) and selecting the image group.



If the image is not uploaded to WDS, we can still retrieve it later in this machine and import it manually to the server using the WDS console.

The capturing process will start and is going to take a while to prepare the reference WIM file.



With that, we've completed the steps to prepare our environment and the reference image is uploaded in WDS.

In the next post, we are going to take a deep dive into the unattended files: Understanding how they work, some best practices and of course generating our own for a complete unattended Windows 8 installation using WDS.

In this post we will complete our step-by-step guide for unattended deployment of Windows 8 images: **Understanding unattended deployment process, generating our own unattended/answer files, reviewing some examples and deploying a full Windows 8 image using our unattended files.**

Understanding Unattended Files

The process of automated and silent deployment of an operating system, as well as for any application, depends on parameterizing correctly our setup. These parameters are stored in what we call **unattended or answer files**.

Starting in Windows Vista / Windows Server 2008 (these two have the same OS kernel), Microsoft improved the setup process of operating systems making it highly scalable and simpler. That's why these unattended files are presented in an **XML**.

Using XML as unattended files makes sense since this markup language is used to structure, store, and transport information. Structure is a key term because Windows setup is composed by “**configuration passes**” which are basically the phases of a Windows installation; and the answer file can structure the information in an efficient manner to offer answers for each phase.

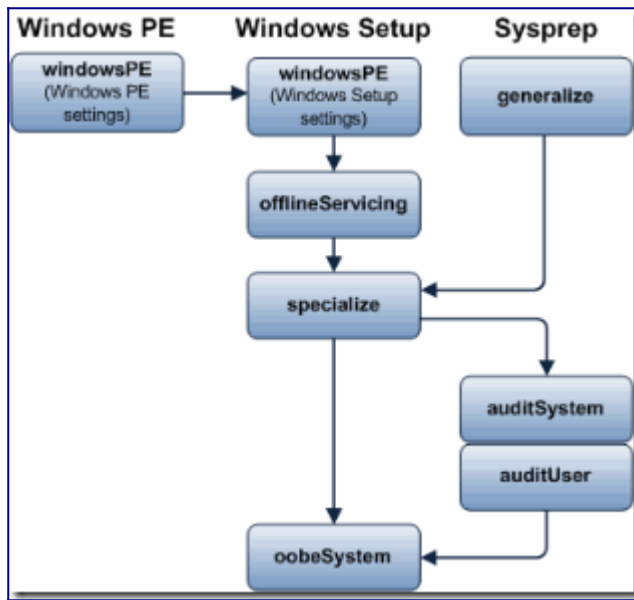
Reviewing Configuration Passes

As we said, the configuration passes represents the phases included in a Windows installation; those phases are basically the same since Windows Vista, even though several of its components have changed over time.

To understand a little bit more about the unattended files we are about to create, let's review the configuration passes included:

1. **windowsPE**: In this phase we can find the configurations and parameters used in Windows Preinstallation environment (reviewed in [Post I](#)), for example handling the disk where we are going to install the OS.
2. **offlineServicing**: This configuration pass is used to apply updates, drivers, or language packs to a Windows image.
3. **generalize**: In this phase computer-specific information is removed from the Windows installation enabling you to capture and reapply the Windows image to different computers.
4. **specialize**: In this stage we can find several Windows configurations like enabling features, network settings and domain settings.
5. **auditSystem**: This phase is only used when we select to boot using audit mode. Settings apply here before a user logs onto the computer, for example to installing out-of-box drivers.
6. **auditUser**: This phase also runs when audit mode is selected in the boot process. Here are settings applied when a user logs onto the computer.
7. **oobeSystem**: In this configuration pass, settings are applied before the “Windows Welcome” message starts. Some options usually used are language or creating user accounts.

Here's a short diagram, taken from Microsoft TechNet, to explain the configuration passes:



For more information, visit the following link: “[How Configuration Passes Work](#)”

Reviewing Unattended Files Best Practices

Before we jump into creating our own unattended files, let’s review some common **best practices** we need to know:

- **Review and understand configuration passes:** We need to have the installation phases clear as we explained above; understanding them will allow us to use the right components and settings.
- **Always validate answer files:** Windows System Image Manager (WISM) is the tool we are going to use to create our unattended files; this tool includes the option for “validation” which verifies that the settings we’ve configured are set correctly and there’s no inconsistency.
- **If you have Windows Vista / Windows Server 2008 unattended files, do not try to use the same for Windows 7 or Windows 8:** Even though all of them use XML files, there have been several changes in time. Take a look to the following link: “[Changes in Unattended Setup Settings from Windows Vista and Windows Server 2008](#)”.
- **Don’t add unnecessary settings:** Answer files could contain hundreds of settings, which translate in time to parse them and slow installation processes. Do not use unnecessary settings that could delay the OS deployment.
- **When you are not sure about the setting value, try WISM help:** All of components include a “help” option which describes the setting and provide some examples. If you are not sure what value to use or if you can leave the setting “empty”, check the help file to verify.
- **Use separate answer files for separate images and architectures:** It is not convenient to use same answer file to different OS architecture. Even though it is possible to include same settings for both architectures in the same file, it could lead us into deployment problems or failures.

6. Preparing Windows System Image Manager (WISM)

Windows System Image Manager (WISM) is included in the [Windows Automated Installation Kit \(AIK\)](#) as one of the tools offered to customize Windows OS deployments.

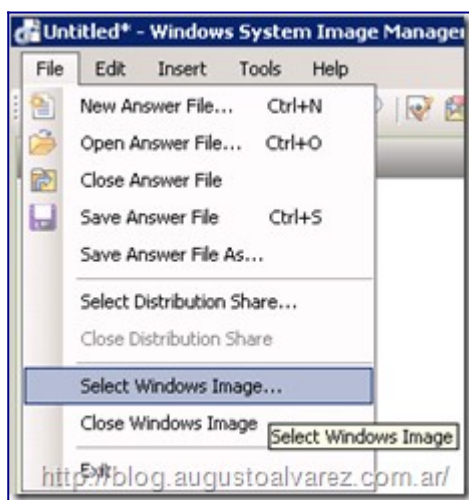
Using WISM we are going to be able, taking a selected OS image, to retrieve the configuration passes included and modify the settings available on each phase. To prepare it for creating our unattended files we only need a simple process:

6.1. Download and install Windows Automated Installation Kit (AIK).

6.2. Copy the “**install.wim**” file from Windows 8 media to a local folder (must be available to WISM for read/write operations).

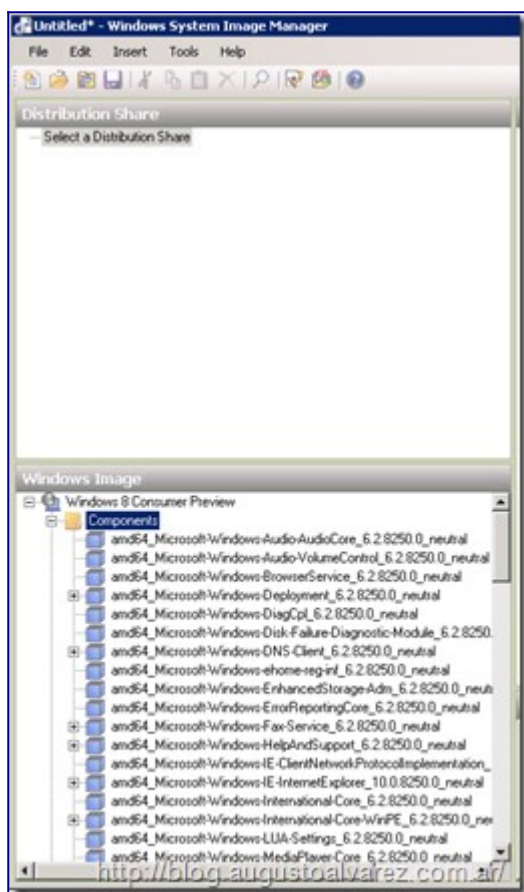
If we don't have the media available, we can still use the reference Windows 8 image we've uploaded in Post I. We need to use the “Export” option from WDS console.

6.3. Access Windows System Image Manager (WISM), “File” and “**Select Windows Image**” and select the WIM file we've copied or exported.



If WISM is not able to perform read/write operations to the file we selected we will receive an error message saying “**Windows SIM was unable to generate catalog. Details: The specified image file did not contain a resource section**”

6.4. Now we will have all the components available in the image. We are going to select several of these components and add those to the configuration passes.



Optionally we can use a “**Distribution Share**” in WISM; in here we can save drivers and other files to use them in the configuration passes.

7. Creating and Using WDSClientUnattend.xml

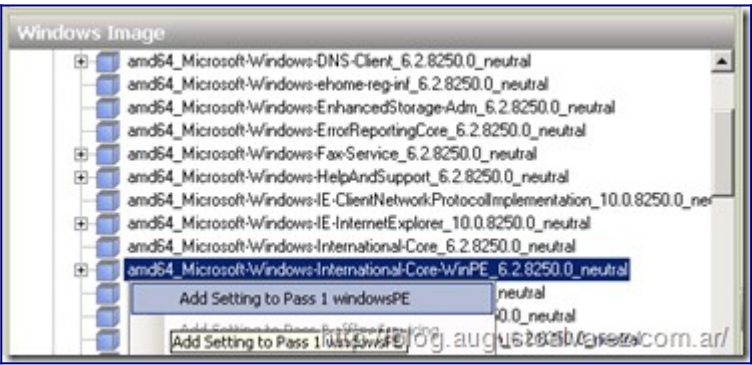
Here’s an [example of WDSClientUnattend.xml for Windows 8](#).

Now that we have WISM ready to start creating answer files, we are going to start with the first one used by WDS: WDSClientUnattend.xml. In this file, we will configure all necessary components related to our first configuration pass: windowsPE.

The components we will need to add are the following:

- *amd64_Microsoft-Windows-International-Core-WinPE_6.2.8250.0_neutral\SetupUILanguage*
- *x86_Microsoft-Windows-Setup_neutral\DiskConfiguration\Disk\CreatePartitions*
- *x86_Microsoft-Windows-Setup_neutral\DiskConfiguration\Disk\ModifyPartitions*
- *x86_Microsoft-Windows-Setup_neutral\WindowsDeploymentServices\ImageSelection\InstallImage*
- *x86_Microsoft-Windows-Setup_neutral\WindowsDeploymentServices\ImageSelection\InstallTo*
- *x86_Microsoft-Windows-Setup_neutral\WindowsDeploymentServices>Login\Credentials*

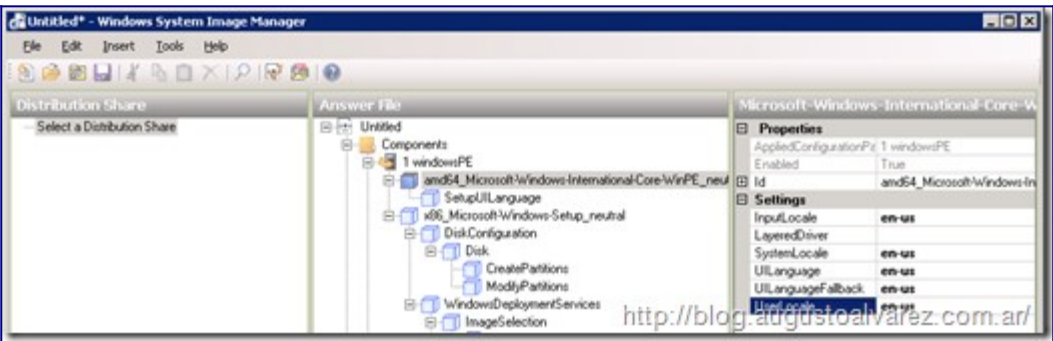
7.1. To start adding them, in “Components” right-click on the selected one and use “Add Setting to Pass 1 windowsPE”.



7.2. After adding all of those mentioned, the WISM console should be looking like this.

Now we need to start editing this components and adding some values.

7.3. For example: Selecting “*amd64_Microsoft-Windows-International-Core-WinPE_6.2.8250.0_neutral*”, we need to configure the options for “*InputLocale*”, “*SystemLocale*”, “*UILanguage*”, “*UILanguageFallback*” and “*UserLocale*”. In my case I’m selecting all of them as “*en-US*”.



To understand the option we are selecting, we can right-click the setting in WISM and select “**Help**”. In there, we will find a complete description to understand the setting and in some cases a few examples to use in the answer file.

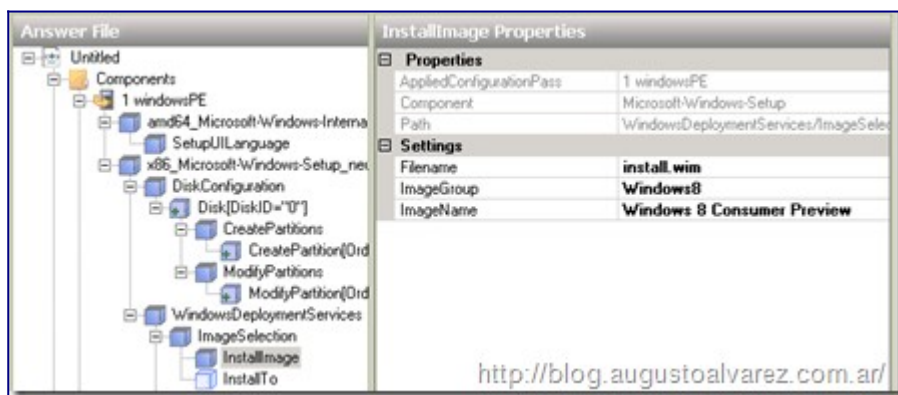
7.4. The rest of values that need to be added can be reviewed in the following table:

WDSClientUnattend		
Component	Setting	Value
amd64_Microsoft-Windows-International-Core-WinPE_6.2.8250.0_neutral	InputLocale	en-us
	SystemLocale	en-us
	UILanguage	en-us
	UILanguageFallback	en-us
	UserLocale	en-us
amd64_Microsoft-Windows-International-Core-WinPE_6.2.8250.0_neutral\SetupUILanguage	UILanguage	en-us
x86_Microsoft-Windows-Setup_neutral\DiskConfiguration\Disk	DiskID	0
x86_Microsoft-Windows-Setup_neutral\DiskConfiguration\Disk\CreatePartitions	Extend	TRUE
	Order	1
	Type	Primary
x86_Microsoft-Windows-Setup_neutral\DiskConfiguration\Disk\ModifyPartitions	Active	TRUE
	Format	NTFS
	Label	Windows
	Letter	C
	Order	1
x86_Microsoft-Windows-Setup_neutral\WindowsDeploymentServices\ImageSelection\InstallImage	PartitionID	1
	Filename	<file name>
	ImageGroup	<group name>
x86_Microsoft-Windows-Setup_neutral\WindowsDeploymentServices\ImageSelection\InstallTo	ImageName	<image name>
	DiskID	0
x86_Microsoft-Windows-Setup_neutral\WindowsDeploymentServices>Login\Credentials	PartitionID	1
	Domain	<domain>
	Password	<password>
http://blog.augustoalvarez.com.ar/	Username	<user name>

Here's an [example of WDSClientUnattend.xml for Windows 8](#).

Notes:

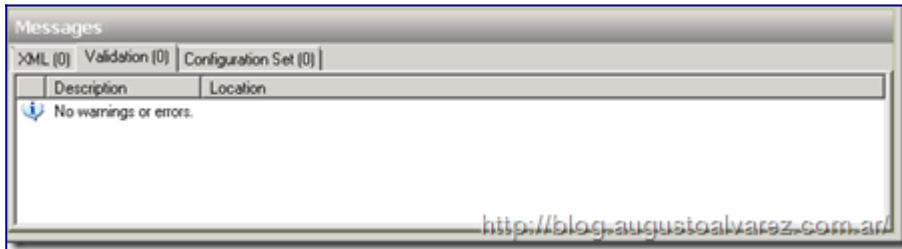
- When we use the “**CreatePartitions**” and “**ModifyPartitions**” components, we need to first right-click on this option, select “**Insert New CreatePartition**” and then we will receive the options to edit.
- Value used in “**Filename**” name must be the WIM file located in WDS. For example “**install.wim**”.
- Also, in “**ImageGroup**” and “**ImageName**” we must use the values used in WDS console.



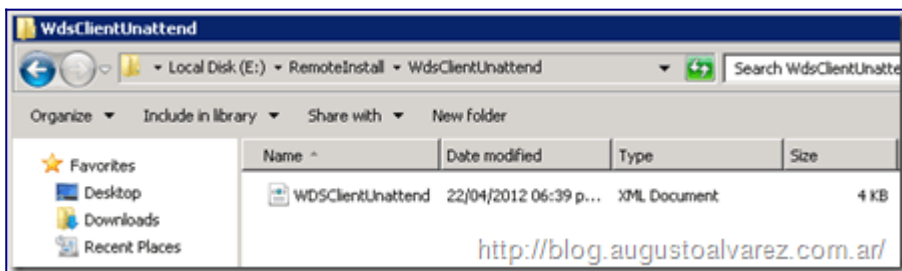
7.5. After completing the settings values, we need to validate the answer file. Select “Tools” and “**Validate Answer File**”.



7.6. Verify that in the lower section, “**Messages**”, there’s no warning / error appearing.

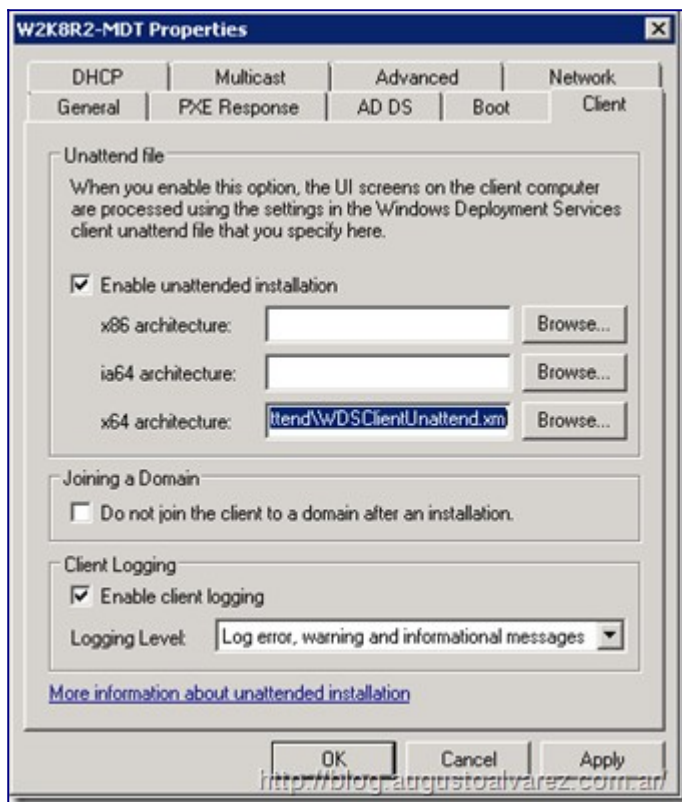


7.7. Save the file and place it in “%drive%\RemoteInstall\WdsClientUnattend”. This particular file must be located in this folder, and should not be moved.



7.8. To configure the unattended file, access the WDS console and right-click the name of the server selecting “**Properties**”.

7.9. Select the “**Client**” pane. Since in this example we’ve used the x64 architecture, browse for the XML file in the selected section.



IMPORTANT: Using WDS we can only assign one WdsClientUnattend file at a time (considering the same architecture for all clients). And as we can see, these file contains the image file we are going to install, so every time we need to change the image, to use a full unattended installation we are going to need to manually change the unattended file.

8. Creating and Using AutoAttend.xml

Here's an [example of AutoAttend.xml for Windows 8](#).

Our second unattended file is dedicated to the Windows customization, as well as providing some important settings to the computer, for example: Product key, computer name, joining it to domain or workgroup, and so on.

Using this answer file we are going to focus in two configuration passes: **4 specialize and 7 oobeSystem**. Let's take a look to the components we are going to use:

Cycle 4: specialize

- *amd64_Microsoft-Windows-UnattendedJoin_neutral\Identification\Credentials*
- *x86_Microsoft-Windows-Shell-Setup_neutral*

Cycle 7: oobeSystem

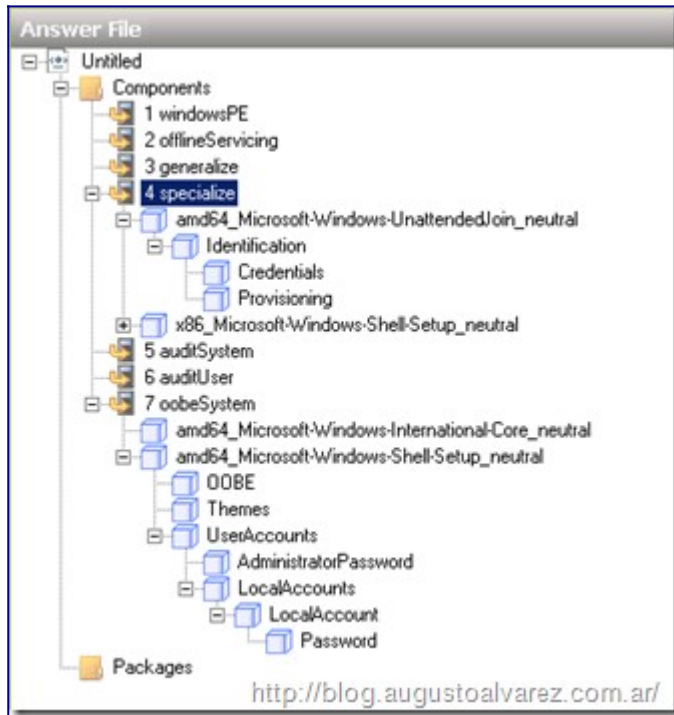
- *amd64_Microsoft-Windows-International-Core_neutral*
- *amd64_Microsoft-Windows-Shell-Setup_neutral\OOBE*
- *amd64_Microsoft-Windows-Shell-Setup_neutral\Themes*
- *amd64_Microsoft-Windows-Shell-Setup_neutral\UserAccounts\AdministratorPassword*

- ***amd64_Microsoft-Windows-Shell-Setup_neutral\UserAccounts\LocalAccounts\LocalAccount\Password***

8.1. Select “File”, “**New Answer File**” WISM to start creating the new unattended file.

8.2. Add the mentioned components to their particular cycles.

8.3. After it’s done, the WISM pane should look something like this.



8.4. Complete the settings using the values shown in the following table.

<http://blog.augustoalvarez.com.ar/> **AutoAttend**

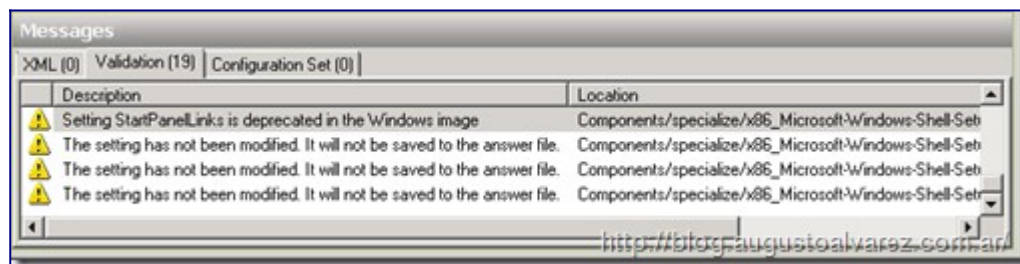
Component	Setting	Value
<i>Cycle 4: specialize</i>		
amd64_Microsoft-Windows-UnattendedJoin_neutral\Identification	JoinDomain	<domain>
amd64_Microsoft-Windows-UnattendedJoin_neutral\Identification\Credentials	Domain	<domain>
	Password	<password>
	Username	<user name>
x86_Microsoft-Windows-Shell-Setup_neutral	ProductKey	<product key>
	TimeZone	SA Eastern Time
<i>Cycle 7: oobeSystem</i>		
amd64_Microsoft-Windows-International-Core_neutral	InputLocale	en-us
	UILanguage	en-us
	UserLocale	en-us
amd64_Microsoft-Windows-Shell-Setup_neutral\OOBE	HideEULAPage	TRUE
	NetworkLocation	Work
	ProtectYourPC	1
amd64_Microsoft-Windows-Shell-Setup_neutral\Themes	DefaultThemesOff	TRUE
amd64_Microsoft-Windows-Shell-Setup_neutral\UserAccounts\AdministratorPassword	Value	<password>
amd64_Microsoft-Windows-Shell-Setup_neutral\UserAccounts\LocalAccounts\LocalAccount	Description	<description>
	DisplayName	Admin
	Group	Administrators
	Name	Admin
amd64_Microsoft-Windows-Shell-Setup_neutral\UserAccounts\LocalAccounts\LocalAccount\Password	Value	<password>

Here's an [example of AutoAttend.xml for Windows 8](#).

Notes:

- In “**ProductKey**” setting, the value must be entered using the “-” as separator between 5 digits. For example: **6RH4V-HNTWC-JQKG8-RFR3R-36498**
- When we add “LocalAccount” component, as we did for disk partitions, we need to right-click the component and select “**Insert New LocalAccount**”. In my example, I’m adding the “Admin” user in the Administrators group.

8.5. Validate the answer file. Take note that using these components, there will be some warnings generated that we can actually ignore.

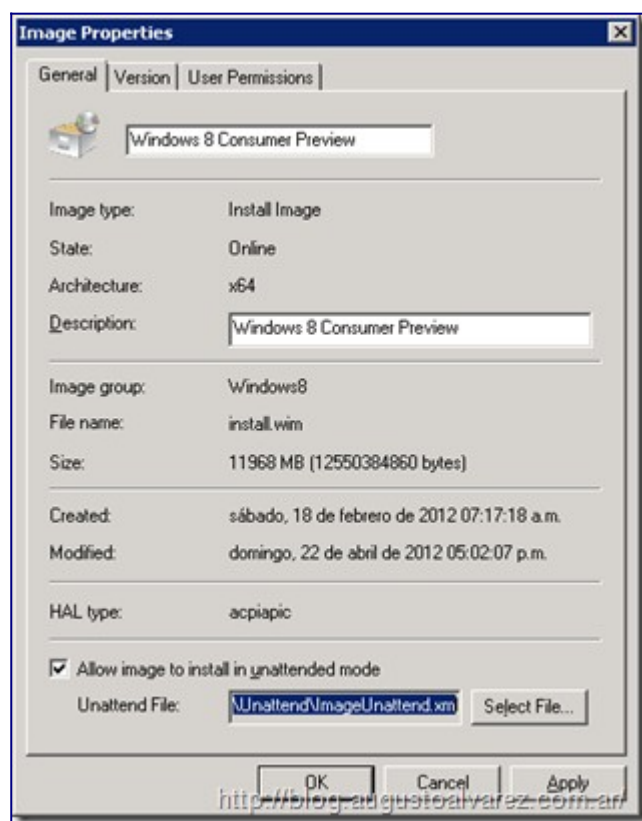


For example, some components that is deprecated and no longer used in Windows images like “StartPanelLinks”.

8.6. Save the answer file and place it in any location available for WDS Server. This particular file does not have to be saved with any special name nor location.

8.7. To associate this file with the reference Windows 8 image, access the WDS console and select **“Properties”** in the install image we would like to use the unattended file.

8.8. In the lower section, select the option **“Allow image to install in unattended mode”** and select the file we’ve just created.

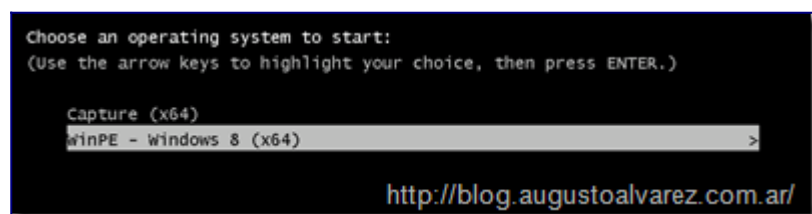


9. Deploying Windows 8 Using Unattended Files

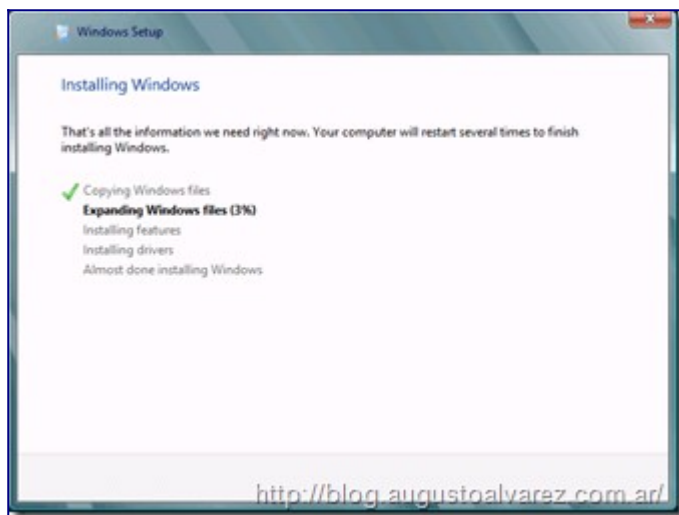
Once we have completed the unattended files and associate them in Windows Deployment Services (WDS) console, the rest is really simple: Just turn on a client machine and start a PXE boot.

9.1. Start the PXE boot in any client machine.

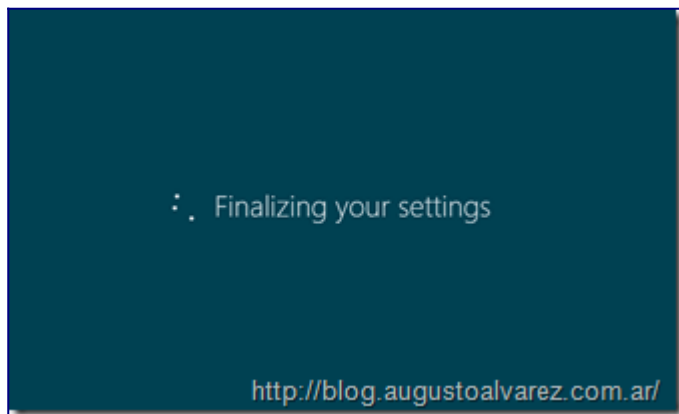
9.2. Make sure you select the Windows PE for booting an installation and not the capture process.



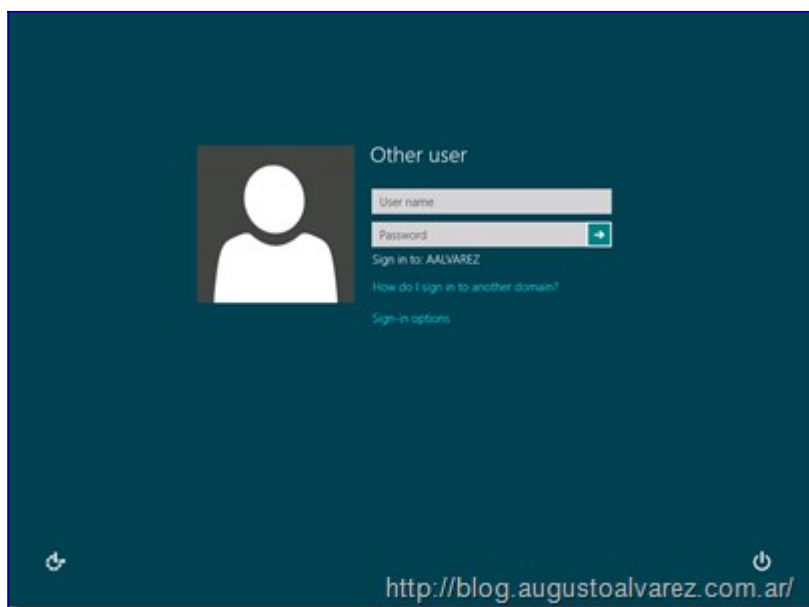
9.3. Review that the steps are completing without any user intervention.



When the normal installation is finished, in “Finalizing your settings” stage all the customizations in the image will be performed.



9.4. After the process is done, we should see in our case the OS ready for account login using domain credentials.



Conclusions

As we can see, the processes involved for a **fully unattended deployment of Windows 8 are really simple**:

- Installing and configuring Windows Deployment Services (WDS) only requires a couple of wizards.
- Adding clean boot and install images for Windows 8 to WDS does not require any complexity, just by using Windows media we can complete it.
- Capturing a reference Windows 8 basically requires running sysprep and boot the machine using a capture boot image.
- To create our own unattended files using WISM, we have all the support we need in the same tool.

Taking these aspects into account, I think all IT departments should consider using an automated and unattended deployment for Windows operating systems. Using this free tools offered by Microsoft can improve IT processes efficiency in large amounts.